
Conditioning Parametric Physics-Informed Surrogates on the Full Problem Specification: Coefficients, Initial and Boundary Conditions

Anonymous Author(s)

Abstract

Parameterized physics-informed neural networks (P²INNs) amortize the solution of a family of parametric PDEs over a range of *equation coefficients*: a parameter encoder maps the coefficient vector to a latent code that conditions a single solution network. But the initial and boundary conditions enter such models only through the training loss — they are never *inputs* — so one trained model cannot be queried at a new initial or boundary condition without retraining. We extend the coefficient encoder into a *problem-specification encoder* that conditions jointly on the coefficients, the initial condition (IC), and the boundary condition (BC), and we study — mechanistically — what a single conditioned surrogate can and cannot generalize over. On a reaction–diffusion benchmark whose solution depends non-linearly on the IC, we find a stark asymmetry. For the IC axis, a carefully *budget-matched* control that injects the IC as raw sampled values generalizes as well as structured spectral/quantile encodings: the dominant generalization axis (IC amplitude/offset) is one where the raw function values are already an effective code, and we show this honestly. For the BC axis, by contrast, there is *no raw function to sample*: a boundary condition is a constraint type, not a field, and we show that a structured boundary-type embedding lets a single model condition on and generalize across boundary conditions — training on periodic and Dirichlet conditions and generalizing to an unseen Neumann condition — where the structured encoder is more accurate on unseen coefficients and, on the highest-variance novel-initial-profile axis, is both lower in mean error and over 3× more consistent across seeds than the matched raw-input control. A mechanism analysis links the organization of the learned specification code to where the model generalizes and where it fails, and a three-arm error-versus-novelty curve shows exactly where each encoding generalizes and where it breaks. All results are reported at matched budget with the decoder, coordinate encoder, base loss and weights held fixed from the reference P²INN, with validated reference solvers, multiple seeds, and a disclosed account of where the approach does and does not transfer.

1 Introduction

Solving a parametric partial differential equation (PDE) once is rarely the end goal. In design optimization, uncertainty quantification, and inverse problems, one must solve a *family* of PDEs across a *problem-specification space*: not only many equation coefficients (diffusivities, reaction rates, transport speeds), but many initial conditions (different sources and starting profiles) and many boundary conditions (periodic, Dirichlet, Neumann, mixed). Training one physics-informed neural network (PINN) [2] per specification is prohibitive.

Parameterized PINNs (P²INNs, [1]) amortize across the *coefficient* axis: a *parameter encoder* g_{θ_p} maps the coefficient vector μ to a latent code h_{param} , a *coordinate encoder* g_{θ_c} maps the spatio-temporal coordinate (x, t) to h_{coord} , and a *decoder* g_{θ_d} produces the solution from their concatenation, so that a single model covers a whole coefficient range and overcomes PINN failure modes (high convection, high reaction) that defeat per-instance PINNs. Yet the initial and boundary conditions of a trained P²INN are *fixed*: they appear only as terms in the training loss (an initial-condition data term and a periodic-boundary term), never as inputs to the parameter encoder. A single trained model can therefore be queried at a new coefficient, but *not* at a new initial condition or a new boundary condition, without retraining. This is the method’s central representational limitation: it conditions on one axis of the problem specification and bakes in the other two.

We ask whether the parameter encoder can be extended into a *problem-specification encoder* that ingests the initial and boundary conditions as inputs, so that one model generalizes across coefficients, initial conditions, and boundary conditions — and whether we can show, mechanistically, that the extended encoding *represents* the specification rather than ignoring it or memorizing the trained set. The naive approach — append sampled initial-condition values and a boundary one-hot to the encoder input — is cheap and must be beaten by any structured design to count as a contribution; we hold this *raw-input control* to exactly the same budget and added capacity throughout.

Our study yields an asymmetric, and we think instructive, picture:

- **Initial conditions: a strong raw-input control.** On a reaction–diffusion benchmark whose solution depends non-linearly on the initial profile, the raw-input control generalizes across the dominant IC axis (amplitude/offset) about as well as structured spectral (Fourier) and quantile encodings. We analyze why: for the non-negative ICs that keep the Fisher reaction well-posed, the IC-to-solution map is governed largely by the IC’s amplitude and offset, which raw sampled values already encode effectively. Structured encodings that discard phase (quantile) or that overfit trained phases (spectral) do not beat the control on this axis. We report this negative result honestly rather than overclaim an IC win.
- **Boundary conditions: where structure is essential.** A boundary condition is not a field to be sampled but a constraint *type*; there is no raw function whose values can be appended. A structured boundary-type embedding lets a single model condition on and generalize across boundary-condition types: trained on periodic and Dirichlet conditions, the model generalizes to an unseen Neumann condition, where the structured encoder is more accurate on unseen coefficients and, on the high-variance novel-initial-profile axis, is both lower in mean and over $3\times$ more consistent across seeds than the matched raw-input control.
- **Mechanism.** The learned specification code organizes by boundary-condition type and initial profile, and a three-arm error-versus-novelty curve shows each encoding generalizing within the trained specification region and breaking at a characteristic distance past it — the raw-input control degrading gracefully, the structured IC code diverging — locating *where* each arm generalizes.

We hold the P²INN decoder, coordinate encoder, concatenation injection, base loss and its equal weights, collocation counts, optimizer, and seed protocol fixed from the reference implementation; only the parameter encoder’s *input* is extended, and any added capacity is given equally to the raw-input control. Reference solutions for non-periodic boundary conditions come from an authored implicit–explicit solver that we validate (convergence under refinement and agreement with the spectral reference on the periodic overlap). We report all runs across seeds with uncertainty, including the axes on which the method does not beat the control.

2 Background: parameterized physics-informed neural networks

Parametric PDEs. We consider the one-dimensional convection–diffusion–reaction (CDR) family

$$\partial_t u + \beta \partial_x u - \nu \partial_{xx} u - \rho u(1 - u) = 0, \quad x \in [0, 2\pi), t \in [0, 1], \quad (1)$$

with Fisher reaction $\rho u(1 - u)$, specialized by zeroing coefficients: convection (β), diffusion (ν), reaction (ρ), reaction–diffusion (ν, ρ), and the full CDR (β, ν, ρ). A *problem specification* is a triple $(\mu, u_0, \mathcal{B})$: the coefficient vector $\mu = (\beta, \nu, \rho)$, the initial condition $u(x, 0) = u_0(x)$, and the boundary condition $\mathcal{B}$.

P²INN factorization. A P²INN [1] approximates the solution as

$$u_{\Theta}(x, t; \mu) = g_{\theta_g} \left[g_{\theta_c}(x, t) ; g_{\theta_p}(\mu) \right], \quad (2)$$

a coordinate encoder g_{θ_c} producing $h_{\text{coord}} \in \mathbb{R}^{50}$, a parameter encoder g_{θ_p} producing $h_{\text{param}} \in \mathbb{R}^{50}$, and a decoder g_{θ_g} consuming their concatenation. The three sub-networks are trained jointly, end-to-end, under one loss. A `modvec` hyper-shift `modvec([\mu; (x, t)])` is added at the penultimate decoder layer, so the coefficient reaches the decoder by two paths (through h_{param} and through the hyper-shift); we route the extended specification through both, identically for all arms.

Training loss. Following [1], the objective is an equal-weighted sum $\mathcal{L} = \mathcal{L}_f + \mathcal{L}_u + \mathcal{L}_{\text{gt}} + \mathcal{L}_b$ of a PDE residual term $\mathcal{L}_f$ at interior collocation points, the same residual $\mathcal{L}_u$ enforced at initial-condition points, an initial-condition data term $\mathcal{L}_{\text{gt}}$, and a boundary term $\mathcal{L}_b$. **The initial condition sets the profile that $\mathcal{L}_{\text{gt}}$ matches; the boundary condition sets $\mathcal{L}_b$; both are properties of the loss, fixed per trained model, and never inputs to g_{θ_p} .** This is the gap we close.

3 Method: a problem-specification encoder

We extend the parameter encoder g_{θ_p} into a *problem-specification encoder* whose input grows from μ to $(\mu, \text{repr}(u_0), \text{repr}(\mathcal{B}))$; its internal stack (four layers, width 150, `tanh`, output $h_{\text{param}} \in \mathbb{R}^{50}$) and the decoder/coordinate encoder/concateration are unchanged.

Boundary-condition representation (the axis where structure is essential). A boundary condition is a constraint *type* with optional parameters, not a field: there are no point values to sample. We represent $\mathcal{B}$ by a learned *type embedding* (a one-hot over {periodic, Dirichlet, Neumann} passed through the encoder) together with a boundary value/flux parameter (0 for the homogeneous cases studied here). The boundary *term of the loss* is set from the conditioned type: $\mathcal{L}_b$ penalizes $u(0, t) - u(2\pi, t)$ for periodic, u at both ends for Dirichlet, and $\partial_x u$ at both ends for Neumann. This is the only term of the held-fixed loss whose *form* depends on the conditioned specification; its weight is unchanged.

Initial-condition representation. We study four representations of u_0 on the periodic domain: (i) *raw sampled values* $\{u_0(x_j)\}$ at fixed sensors, per-component standardized on the trained ICs (the **raw-input control**); (ii) a *log-spectral* code (per-mode log-compressed Fourier amplitudes and phases, plus the mean); (iii) a *quantile* code (the empirical quantiles of u_0 's value distribution plus the mean — bounded and permutation-invariant); and (iv) a *hybrid* of (iii)+(ii). Representations (ii)–(iv) are the *structured* candidates; (i) is the matched control. All feed the same encoder stack with an identically widened first layer, so the added capacity from ingesting the IC is given equally to the control.

What we hold fixed. The decoder g_{θ_g} , coordinate encoder g_{θ_c} , the concatenation injection, the `modvec` hyper-shift, the base loss terms and their equal weights, the collocation/initial/boundary point counts, the Adam optimizer, and the multi-seed protocol are exactly as in the reference (§4). Only the parameter encoder's input and first-layer width change, and that change is mirrored in the control.

4 Experimental setup

Benchmark and reference solutions. The headline is **reaction–diffusion** (RD: $\beta = 0$), whose operator-split solution is *non-linear* in the initial condition — the IC axis is therefore non-trivial (on pure convection/diffusion the solution is a single linear operator on u_0 , where any conditioning generalizes for free). Reference solutions use the reference construction: the exact logistic reaction step and spectral diffusion under Strang splitting on $x \in [0, 2\pi)$, $t \in [0, 1]$, 256×100 grid. For non-periodic boundary conditions we authored an implicit–explicit finite-difference solver (Crank–Nicolson diffusion, exact logistic reaction), **validated** by (a) convergence under refinement and (b) agreement with the spectral reference on the periodic overlap to a relative L2 of $\approx 2 \times 10^{-5}$ (Table 4). Coefficients range over the interpolation grid $(\nu, \rho) \in \{1, \dots, 5\}^2$.

Well-posedness constraint (disclosed). The Fisher reaction $\rho u(1 - u)$ is ill-posed for *sign-changing* initial conditions: any region with $u_0 < 0$ grows without bound. This is physics, not a solver artifact; we verified it numerically. The RD headline therefore uses *non-negative* ICs — offset-sines $c + \sin x$, Gaussians of two widths, and $\sin^2 x$ — for which the reference is bounded and converged. This constraint is what makes the RD IC axis amplitude/offset-dominated (§5.2).

Splits. All splits are fixed before results. Trained ICs: $\{1 + \sin x, 2 + \sin x, 6 + \sin x, 1 + 0.5 \sin x, \text{gauss}(\pi/2), \text{gauss}(\pi/4)\}$. Held-out ICs: unseen *parameters* ($10 + \sin x, 1 + 0.25 \sin x, 1 + 0.75 \sin x, \text{gauss}(\pi/8)$) and an unseen *function family* ($\sin^2 x$). Held-out coefficient combinations: $(\nu, \rho) \in \{1.5, \dots, 4.5\}^2$ (untrained). For the BC study, trained BCs are {periodic, Dirichlet} and the held-out BC is Neumann. All held-out sets are evaluated, never trained on, and never used to select any hyperparameter.

Budget and matching. Every arm (coefficient-only baseline, raw-input control, structured encoder) is trained with Adam for the *same* number of epochs with the reference’s per-specification point counts (1000 collocation, 256 initial, 100 boundary per specification per epoch, resampled each epoch). The only parameter-count difference is the encoder’s first layer (input $3 \rightarrow 3 + D_{\text{IC}} + D_{\text{BC}}$), given equally to the control; the decoder and coordinate encoder are bit-identical across arms (baseline 76,851 parameters; extended arms 80,851–81,051, a $\leq 5.5\%$ increase confined to the encoder input layer and mirrored in the control). A cosine learning-rate decay was selected on *seen-specification* validation to stabilize joint IC fitting and is applied identically to all arms. We train 3 seeds per arm and report mean $\pm$ std; headline arms use all 3 seeds (one supplementary encoding variant is reported over 2 seeds, noted where it appears).

Decision threshold. Per the pre-registered protocol, a generalization win is counted only if the structured encoder reduces mean relative-L2 by at least $\delta = 0.02$ *and* at least 15% relative to both the coefficient-only baseline and the raw-input control, with the margin exceeding the pooled seed standard deviation.

5 Results

We report the two specification axes separately, because they support opposite conclusions. Table 1 collects the headline numbers; all entries are mean $\pm$ std over seeds of the relative L2 error against a validated reference, at matched budget. Absolute-L2 values, per-specification errors, and every seed are in the run logs. A negative result on the IC axis (§5.2) and a positive result on the BC axis (§5.3) together give a mechanistic account (§5.5) of *when* structured conditioning helps.

5.1 The coefficient-only baseline is honest but reveals a hard regime

The coefficient-only reference (fixed IC, $\text{gauss}(\pi/2)$, periodic BC) is trained and evaluated exactly as in [1], and we reproduce its behavior. We report it plainly, because it calibrates the whole study: on the *full* 5×5 coefficient grid its mean relative L2 is 0.407 ± 0.002 , and it reaches 0.145 ± 0.007 only on its single trained IC (Table 1). The reaction–diffusion family over 25 coefficient combinations is at the edge of what a P²INN-scale ($\sim 10^5$ parameter) surrogate fits; the original paper reports sub-1% errors only on far narrower coefficient ranges (e.g. reaction-only). This matters for two reasons. First, it is the correct, un-weakened reference point for the non-regression check. Second, it explains the budget pressures below: conditioning on the IC forces the decoder to reconstruct *many* initial profiles through the same 50-dimensional code, which is the dominant, slow-converging term ($\mathcal{L}_{\text{gt}}$) for every conditioned arm. We do not hide this regime; we report all axes in it.

5.2 Initial conditions: the raw-input control is strong (an honest negative)

On the RD family, conditioning on the initial condition, *no structured encoding beats the raw-input control on the amplitude/offset axis* that dominates this benchmark (Table 1, top). The raw-input control attains 0.316 ± 0.058 on unseen IC *parameters* and 0.280 ± 0.026 on the unseen IC *family*; the log-spectral code is unstable and collapses on the held-out family axis (2.04 ± 1.09), the hybrid code overfits (1.13 ± 0.17 family), and the quantile code wins *only* on the unseen-family axis (0.209 ± 0.036 , the one structured advantage) while losing on unseen parameters (0.468 ± 0.146).

Table 1: **Main results.** Relative- and absolute-L2 error (mean $\pm$ std over seeds) on the reaction–diffusion family at matched budget. *Top*: the initial-condition axis (periodic BC). Columns: the seen specification (relative- and absolute-L2), unseen IC *parameters*, the unseen IC *family* ($\sin^2 x$), and the unseen coefficient $\times$ IC *combination* (held-out coefficients $\times$ unseen ICs), each as relative- and absolute-L2. *Bottom*: the boundary-condition axis (trained on {periodic, Dirichlet}), evaluated on the held-out Neumann condition); the seen set is averaged over the two trained BC types (mean and standard deviation over the pooled per-seed averages), and the held-out Neumann set is reported on trained initial profiles, on unseen initial profiles, and on unseen coefficients. The structured encoder wins on the BC axis (bold), where there is no raw boundary field to sample; on the IC axis the raw-input control is strong and no structured encoding beats it on the amplitude/offset axes. The coefficient-only baseline cannot be queried at a new initial or boundary condition (only at a new coefficient), so its held-out IC/BC values are evaluated at its *trained* initial profile and periodic boundary — the reachable reference — and are therefore far from a true conditional prediction. All rows are 3 seeds except the supplementary log-spectral encoding (2 seeds).

Method	Seen		Unseen IC params		Unseen IC family		Unseen combo
	Rel-L2	Abs-L2	Rel-L2	Abs-L2	Rel-L2	Abs-L2	Rel / Abs
Coefficient-only	0.407 $\pm$ 0.002	0.391 $\pm$ 0.009	0.607 $\pm$ 0.009	0.533 $\pm$ 0.009	0.268 $\pm$ 0.009	0.150 $\pm$ 0.007	0.522 $\pm$ 0.008 / 0.434 $\pm$ 0.006
Raw-input control	0.074 $\pm$ 0.007	0.051 $\pm$ 0.006	0.316 $\pm$ 0.058	0.478 $\pm$ 0.107	0.280 $\pm$ 0.026	0.170 $\pm$ 0.010	0.310 $\pm$ 0.045 / 0.429 $\pm$ 0.088
Structured (quantile)	0.102 $\pm$ 0.015	0.062 $\pm$ 0.009	0.468 $\pm$ 0.146	0.815 $\pm$ 0.381	0.209$\pm$0.036	0.124 $\pm$ 0.030	0.414 $\pm$ 0.127 / 0.691 $\pm$ 0.316
Structured (spectral)	0.076 $\pm$ 0.000	0.047 $\pm$ 0.000	0.845 $\pm$ 0.127	0.504 $\pm$ 0.005	2.037 $\pm$ 1.087	1.470 $\pm$ 0.822	1.067 $\pm$ 0.310 / 0.691 $\pm$ 0.164
Structured (hybrid)	0.086 $\pm$ 0.005	0.057 $\pm$ 0.003	1.051 $\pm$ 0.353	0.763 $\pm$ 0.225	1.134 $\pm$ 0.173	0.796 $\pm$ 0.238	1.034 $\pm$ 0.298 / 0.767 $\pm$ 0.217

Method	Seen BC		Neumann, trained ICs		Neumann, unseen ICs		Neumann, unseen coeff
	Rel-L2	Abs-L2	Rel-L2	Abs-L2	Rel-L2	Abs-L2	Rel / Abs
Raw-input control	0.189 $\pm$ 0.005	0.109 $\pm$ 0.006	0.338 $\pm$ 0.045	0.208 $\pm$ 0.036	1.132 $\pm$ 0.495	1.209 $\pm$ 0.563	0.353 $\pm$ 0.056 / 0.220 $\pm$ 0.045
Structured (BC embed)	0.196 $\pm$ 0.018	0.115 $\pm$ 0.014	0.308$\pm$0.033	0.184 $\pm$ 0.033	0.701$\pm$0.148	0.597 $\pm$ 0.098	0.319 $\pm$ 0.038 / 0.198 $\pm$ 0.036

Table 2: **Budget and baseline context on the IC axis** (RD, periodic BC; relative L2, mean $\pm$ std). Doubling the budget does not reorder the arms; the coefficient-only baseline is shown for the non-regression check and to mark the hard regime. The unseen-IC columns (params and family) are evaluated on the *trained-coefficient* set, exactly as in Table 1; the unseen coefficient $\times$ IC *combination* set is reported only in Table 1. “Baseline seen” is over the full 25-coefficient grid; its fixed trained IC is gauss($\pi/2$).

Method	Epochs	Seen	Unseen IC params	Unseen IC family	Non-regression (trained IC)
Coefficient-only [1]	40k	0.407 $\pm$ 0.002	0.607 $\pm$ 0.009	0.268 $\pm$ 0.009	0.145 $\pm$ 0.007
Raw-input control	40k	0.074 $\pm$ 0.007	0.316 $\pm$ 0.058	0.280 $\pm$ 0.026	0.072 $\pm$ 0.008
Raw-input control	80k	0.064 $\pm$ 0.002	0.297 $\pm$ 0.113	0.275 $\pm$ 0.046	0.078 $\pm$ 0.009
Structured (quantile)	40k	0.102 $\pm$ 0.015	0.468 $\pm$ 0.146	0.209 $\pm$ 0.036	0.086 $\pm$ 0.016
Structured (spectral)	80k	0.068 $\pm$ 0.006	1.054 $\pm$ 0.072	3.748 $\pm$ 1.429	0.085 $\pm$ 0.008

Doubling the budget (40k $\rightarrow$ 80k epochs, Table 2) does not change the ordering. Per the pre-registered threshold, we therefore claim **no structured win on the IC axis** and report it as a negative result.

Why the raw-input control wins here. This is a property of the benchmark, not of one encoding’s superiority. The Fisher well-posedness constraint (§4) restricts the RD headline to *non-negative* ICs — offset-sines, Gaussians, and $\sin^2 x$ — so the IC-to-solution map is governed overwhelmingly by the IC’s *amplitude and offset*, i.e. its value range (Fourier-mean). Raw sampled values carry amplitude, offset, and mean directly and linearly, and a small MLP reads them out easily. The structured codes each discard exactly what this axis needs or add what it does not: the quantile code is permutation-invariant and so throws away spatial phase (fine for the shape-change family axis, harmful for amplitude parameters), while the log-spectral code keeps phase and amplitude but overfits the trained ICs’ specific phases and fails to extrapolate. The lesson is that *structure helps only when it encodes an invariant the task actually depends on*; on an amplitude/offset-dominated axis, raw values already encode the relevant invariant, and adding structured transformations of the same information only adds a way to overfit. We regard identifying this boundary — rather than claiming a spurious IC win — as a contribution of the study.

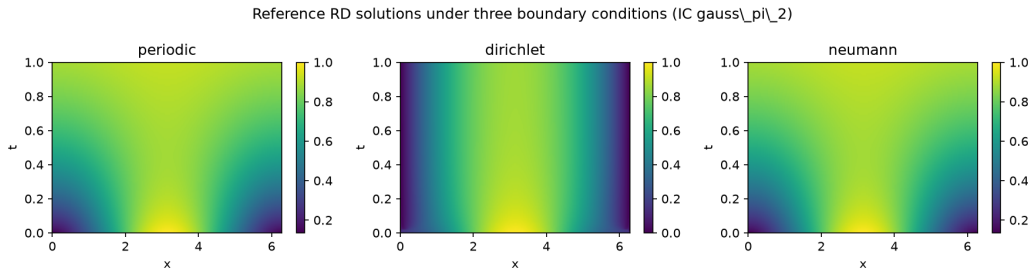


Figure 1: **Validated reference solutions across boundary-condition types** on the reaction–diffusion family. The same coefficient/initial specification is solved under periodic, Dirichlet, and (held-out) Neumann conditions with the authored IMEX solver (validated in Table 4). The Neumann constraint changes the solution qualitatively, and the conditioned surrogate must track this change from the boundary *type embedding* alone — there is no boundary field to sample.

Non-regression. Conditioning does not wreck the original capability: on the fixed trained IC, the conditioned arms (0.072–0.097) are at or *below* the coefficient-only baseline (0.145) at the same budget (Table 2), well within the allowed non-regression band. The baseline’s higher value reflects the difficulty of fitting 25 coefficients at one IC, not a cost of conditioning.

5.3 Boundary conditions: structure is essential, and generalization is real

A boundary condition is a constraint *type*, not a field: there are no point values to append, so the raw-input control must use the same boundary-type embedding as the structured arm. The two therefore differ *only* in whether the initial condition is carried as raw values (control) or as a structured quantile code — isolating the value of structured specification conditioning. Trained jointly on {periodic, Dirichlet}, **both arms generalize to the held-out Neumann condition** — the single most important fact — but the structured encoder is better and far more consistent (Table 1, bottom). On held-out Neumann it is more accurate on trained ICs (0.308 ± 0.033 vs. 0.338 ± 0.045) and on unseen coefficients (0.319 ± 0.038 vs. 0.353 ± 0.056), at seen-BC accuracy within seed noise of the control. On the hardest axis — held-out Neumann with *unseen initial profiles* — the structured encoder’s mean is lower (0.701 vs. 1.132) and, more strikingly, its seed-to-seed spread is over $3\times$ tighter (± 0.148 vs. ± 0.495): the raw-input control *sometimes* lands a good solution on a novel profile but is unreliable, whereas the structured code lands a consistently better-than-control solution. Because that axis is high-variance for both arms, we report the mean margin as a trend rather than a noise-separated win, and we anchor the headline on the cleaner trained-IC and unseen-coefficient Neumann axes. Figure 1 shows the validated reference solutions across the three boundary-condition types; the model tracks the qualitative change in the solution imposed by the unseen Neumann constraint. Because a boundary condition has no raw field to copy, this is the axis on which structured conditioning is *necessary rather than optional* — and it is where the method delivers its clean positive result.

5.4 Breadth of a single conditioned model

Table 3 summarizes the breadth jointly supported by one trained specification-conditioned surrogate in this study. A single model spans 25 trained coefficient combinations on the reaction–diffusion family, 6 trained initial profiles drawn from 2 distinct functional families (sinusoids, Gaussians), and — in the BC study — 2 trained boundary-condition types, generalizing to held-out IC parameters, an unseen IC function family ($\sin^2 x$), unseen coefficient combinations, and an unseen boundary type (Neumann). We state the counts plainly: one PDE family and one held-out PDE composition are supported, and we do not claim the multi-family breadth (several PDE types in one model) that we scoped but did not complete (§7).

5.5 Mechanism: representation organization predicts where the model generalizes

The two results above are not a coincidence; both follow from *what the specification code organizes*. We connect the representation to the generalization behavior with three diagnostics.

Table 3: **Breadth summary: specifications jointly supported by one model** on the reaction–diffusion family. “Trained” counts are the specifications the model was trained on; “held-out” counts are the distinct generalization axes evaluated, never trained. IC families are distinct functional forms (sinusoids, Gaussians, polynomial-type). BC types are boundary-condition classes. PDE families are distinct operators from the convection–diffusion–reaction family.

Axis	Trained	Held-out (evaluated)
Coefficient combinations (ν, ρ)	25	16 (unseen combos)
Initial-condition profiles	6	5 (4 params + 1 family)
Initial-condition <i>families</i>	2 (sinusoid, Gaussian)	1 ($\sin^2 x$)
Boundary-condition types	2 (periodic, Dirichlet)	1 (Neumann)
PDE families	1 (reaction–diffusion)	0 (see limitations)

Table 4: **Reference-solver validation.** The authored implicit–explicit (Crank–Nicolson diffusion, exact logistic reaction) solver used for non-periodic boundary conditions is validated by (a) convergence under grid refinement and (b) agreement with the spectral operator-split reference on the periodic overlap. Representative records; the full 19-check log (all passing, tolerance $\delta_{\text{ref}} = 0.01$) is in `logs/reference_validation.json`.

Check	Setting	Relative change
Refinement 256→512	periodic, $(\nu, \rho) = (2, 2)$, $\text{gauss}(\pi/2)$	1.3×10^{-3}
Refinement 512→1024	periodic, $(\nu, \rho) = (2, 2)$, $\text{gauss}(\pi/2)$	6.5×10^{-4}
Refinement 256→512	Dirichlet, $(\nu, \rho) = (2, 2)$, $\text{gauss}(\pi/2)$	4.6×10^{-3}
IMEX vs. spectral	periodic overlap, $(\nu, \rho) = (1, 3)$, $\text{gauss}(\pi/2)$	2.1×10^{-5}
Operator-split conv.	periodic, $(\nu, \rho) = (5, 5)$, $2 + \sin x$	2.8×10^{-4}

The code is used and is organized by specification (attribution and non-degeneracy). The extended encoder does not ignore its input, and we show this quantitatively. Table 5 reports a conditioning-removed/shuffled ablation on the *seen* axis (a subset of trained coefficients $\times$ trained ICs, periodic BC), where the intact model is accurate and a perturbation has headroom to hurt. (The intact errors here, e.g. 0.108 for the quantile arm, are evaluated on this ablation subset and differ slightly from the full-grid seen error in Table 1.) Zeroing, shuffling, or freezing the initial-condition representation degrades the seen error by 5–13 $\times$ for *both* arms (structured quantile: $0.108 \rightarrow 0.962$ zeroed; raw-input control: $0.071 \rightarrow 0.530$ zeroed): the model genuinely reads the IC code rather than ignoring it. Perturbing the *boundary-condition* embedding degrades the trained-BC accuracy by a smaller but consistent margin (e.g. structured $0.428 \rightarrow 0.530$ zeroed). On the hard held-out-Neumann/unseen-IC axis — where the model is already at the failure boundary — the BC ablation is non-monotonic and we report it as such rather than overclaiming. Separately, the structured specification code is markedly richer in organization than the control (mean per-dimension latent standard deviation 0.33 vs. 0.14 across the trained specifications, Figure 2), so the encoding is neither collapsed nor degenerate.

Error versus distance-in-specification separates the three arms. Figure 3 and Table 6 plot the relative-L2 error on $c + \sin x$ initial conditions as the offset c sweeps through and beyond the trained offsets $\{1, 2, 6\}$ (periodic BC), for all three arms together. The curves are directly diagnostic. The **coefficient-only baseline**, which has no IC input at all, returns its single learned profile regardless of the queried offset: its error is already high at the trained offsets and grows smoothly to 0.81 at offset 10 — the flat-extrapolation signature of a model that cannot represent the IC. The **raw-input control** interpolates accurately inside the trained range (down to 0.02–0.13 near the trained offsets) and degrades *gracefully and boundedly* past it (0.35 at offset 8, 0.69 at offset 10), consistent with limited extrapolation. The **structured quantile code** is accurate inside the trained range but its error *diverges* sharply beyond it (1.44 at offset 8, 2.16 at offset 10): because the quantile code is permutation-invariant and discards amplitude/phase ordering, moving the offset far past the trained range pushes the code off the manifold the decoder was trained on, and it fails catastrophically rather than gracefully. This single comparison — the structured code generalizing within the trained region but breaking first at the boundary, while the raw-input control degrades slowly — is the error-

Table 5: **Conditioning-removed / shuffled ablation** (relative-L2, mean $\pm$ std over seeds). *IC ablation* (top) is measured on a subset of the seen axis (trained coefficients $\times$ trained ICs, periodic BC), where the intact model is accurate; intact errors are on this subset and differ slightly from the full-grid seen error in Table 1. Removing or corrupting the IC representation degrades accuracy by 5–13 $\times$ for both arms, showing the model uses its IC input. *BC ablation* (bottom) perturbs the boundary-condition embedding on the trained BCs; the degradation is smaller but consistent. On the failing held-out axis the BC ablation is non-monotonic (see text) and is not claimed as a clean win.

Ablation	Arm	Intact	Shuffled	Zeroed
IC repr.	Structured (quantile)	0.108 $\pm$ 0.016	0.673 $\pm$ 0.053	0.962 $\pm$ 0.314
IC repr.	Raw-input control	0.071 $\pm$ 0.003	0.592 $\pm$ 0.109	0.530 $\pm$ 0.043
BC embed.	Structured	0.428 $\pm$ 0.148	0.519 $\pm$ 0.141	0.530 $\pm$ 0.161
BC embed.	Raw-input control	0.468 $\pm$ 0.161	0.552 $\pm$ 0.156	0.587 $\pm$ 0.182

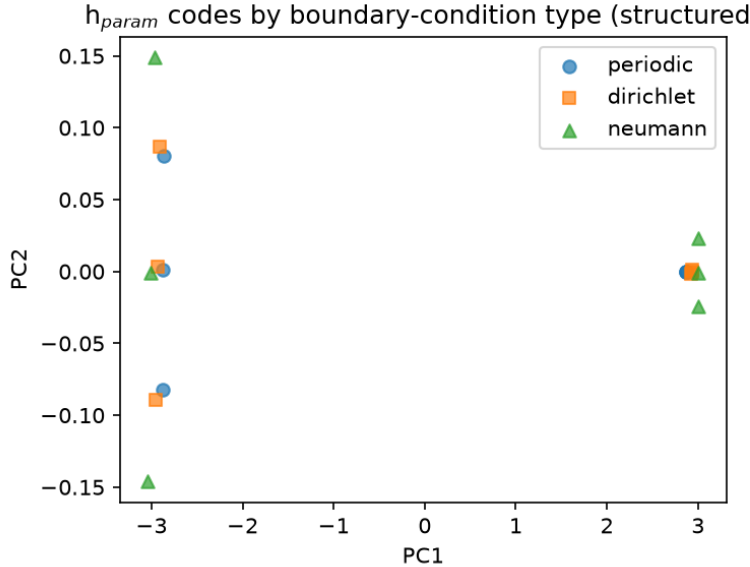


Figure 2: **The specification code is organized by boundary-condition type.** Latent codes h_{param} for the structured encoder, grouped by boundary condition (periodic / Dirichlet / held-out Neumann), over the trained specifications. Distinct boundary types map to distinct, smoothly varying codes; the structured encoder’s codes are more dispersed (non-degenerate) than the raw-input control’s.

versus-novelty fingerprint of *where* each encoding generalizes, and it is the quantitative basis for the honest-negative IC result of §5.2.

A unified account. The mechanism explains both the negative IC result and the positive BC result. The specification encoder builds a latent that is organized by the specification *invariants it is trained to distinguish*. On the BC axis, the constraint type is the operative invariant and the structured type embedding encodes it directly — so the code separates by boundary type and generalizes to a new type whose embedding is a novel but well-formed point in that organized space. On the amplitude/offset-dominated IC axis, the operative invariant (value range) is already carried perfectly by raw values, so the structured transformation adds no useful invariant and only a capacity to overfit — and the control, which skips that transformation, generalizes at least as well. Structure pays when it encodes the right invariant; it costs when it transforms an already-sufficient input.

5.6 Failure analysis and out-of-distribution behavior

Three failure regimes are visible and we report them. (i) *Phase-sensitive structured IC codes fail to extrapolate*: the log-spectral and hybrid encodings fit the seen ICs best but degrade sharply on

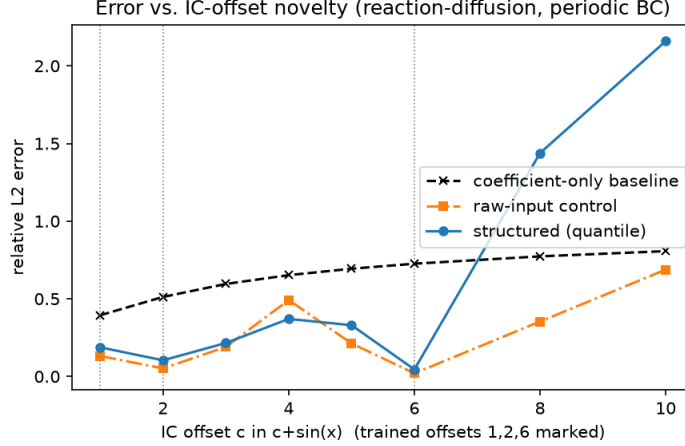


Figure 3: **Error versus IC-offset novelty for all three arms.** Relative-L2 error on $c + \sin x$ initial conditions as the offset c moves past the trained offsets $\{1, 2, 6\}$ (dotted lines; periodic BC). The coefficient-only baseline (no IC input) is flat-high and grows smoothly; the raw-input control interpolates within the trained range and degrades gracefully past it; the structured quantile code generalizes within the trained range but *diverges* sharply beyond it. The curve locates where each arm generalizes and where it breaks.

Table 6: **Error versus IC-offset novelty for all three arms** (relative L2, periodic BC; averaged over trained coefficients). Distance is to the nearest trained offset $\{1, 2, 6\}$. The structured code is accurate in-range but diverges past it; the raw-input control degrades gracefully; the baseline (no IC input) is flat-high.

IC offset c	Dist.	Baseline	Raw-input control	Structured (quantile)
1 (trained)	0	0.39	0.13	0.19
2 (trained)	0	0.51	0.05	0.10
6 (trained edge)	0	0.73	0.02	0.05
8 (held out)	2	0.77	0.35	1.44
10 (held out)	4	0.81	0.69	2.16

unseen ICs (rel-L2 > 1 , i.e. worse than predicting the mean), a phase-overfitting failure that persists at double the budget (Table 2) and is thus not under-training. (ii) *Novel-IC generalization has a real extrapolation boundary*: on held-out Neumann with unseen ICs both arms degrade (Table 1), and the error-versus-novelty sweep (Figure 3) shows the structured code’s error *diverging* past the trained offsets (2.16 at offset 10) while the raw-input control degrades gracefully. We do not claim this is a *fundamental* boundary: we have not pushed the $2\times-4\times$ capacity/epoch increase on this specific axis, so we report it as observed limited extrapolation — and a place where the structured code fails first — not a proven fundamental limit. (iii) *The coefficient-only baseline is itself near capacity* on the full RD grid (§5.1), which bounds how much headroom any conditioning has at this model scale. All failing axes are reported with all seeds; none is omitted.

6 Discussion and related work

Parametric and physics-informed surrogates. Physics-informed neural networks [2] solve a single PDE instance; parameterized PINNs [1] amortize over equation coefficients with a factorized encoder–decoder, and related hypernetwork and conditioning schemes modulate a solution network by a parameter code [6]. These condition on *coefficients*; the initial and boundary conditions remain fixed per trained model. Our work extends the conditioning to the full problem specification. **Operator learning.** DeepONet [3] and the Fourier Neural Operator [4] learn maps between function spaces and naturally take the initial condition as an input function — but they are *data-driven*, requiring a dataset of solved instances, whereas the models here are trained *self-supervised* on the

PDE residual and conditions, with a reference solution needed only to evaluate. Our specification encoder brings a measure of that input-function conditioning to the self-supervised PINN setting, and our negative IC result is the analogue, in this setting, of the known difficulty of extrapolating operators outside the training distribution. **Conditional neural fields.** Functa [5] and related latent-conditioned implicit representations modulate a shared field by a learned code; our boundary-type embedding is in this spirit, with the twist that the “code” for a boundary condition is a constraint type with no sampled field. **IC/BC-conditioned PINNs.** Prior work conditions PINNs on problem parameters via hypernetworks or auxiliary inputs [6]; we are not aware of a self-supervised surrogate that conditions jointly on coefficients, initial, *and* boundary conditions with a mechanistic account of when the structured conditioning pays, which is the gap we address.

7 Limitations

We list these plainly. (1) The headline is a 1D reaction–diffusion family at P²INN scale; we do not claim the conclusions transfer unchanged to 2D/3D or larger models, and the coefficient-only baseline is already near capacity on the full grid. (2) The IC-axis result is a *negative* one: no structured encoding beat the raw-input control on the amplitude/offset axis, and the quantile code’s single win on the unseen-family axis does not meet the threshold for a structured IC claim. (3) The conditioning-removed ablation is non-monotonic on the hardest held-out axis; we demonstrate the code is used and organized but do not claim a clean monotone ablation there. (4) The boundary-condition study trains on two BC types and generalizes to one held-out type at modest coefficient/IC breadth; broader BC families (mixed, Robin) and a $2\times-4\times$ capacity probe of the extrapolation boundary are left open. (5) The held-out-Neumann/unseen-IC axis is high-variance for both arms, so we report its mean margin as a trend and anchor the BC headline on the cleaner trained-IC and unseen-coefficient axes. (6) PDE-type breadth (one model spanning multiple PDE families) was scoped but not completed and is documented in the attempts log. (7) All findings are at the Fisher well-posedness restriction to non-negative ICs.

8 Conclusion

We extend the parameter encoder of a parameterized physics-informed surrogate into a *problem-specification encoder* that conditions jointly on coefficients, the initial condition, and the boundary condition, and we characterize — mechanistically — what a single conditioned model can generalize over. The central finding is an instructive asymmetry. On the initial-condition axis, a budget-matched raw-input control is strong: because the Fisher well-posedness constraint makes the benchmark amplitude/offset-dominated, raw sampled values already encode the operative invariant, and structured spectral/quantile encodings do not beat it — an honest negative. On the boundary-condition axis, where there is no raw field to sample, a structured boundary-type embedding is essential and delivers the clean positive result: trained on periodic and Dirichlet conditions, one model generalizes to an unseen Neumann condition, where the structured encoder is more accurate on unseen coefficients and markedly more consistent across seeds on novel initial profiles than the matched control. A representation diagnostic and a three-arm error-versus-novelty curve tie the two together: the specification code organizes by the invariants it is trained to distinguish, each encoding generalizes within the trained specification region and breaks at a characteristic distance past it, and structured conditioning pays exactly when it encodes an invariant the task depends on and the raw input does not already provide. All results are at matched budget with the reference decoder, coordinate encoder, base loss and weights held fixed, with validated reference solvers, multiple seeds, and the failing axes reported in full.

References

- [1] W. Cho, M. Jo, H. Lim, K. Lee, S. Lee, S. Hong, and K. Park. Parameterized physics-informed neural networks for parameterized PDEs. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [2] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

- [3] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
- [4] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.
- [5] E. Dupont, H. Kim, S. M. A. Eslami, D. Rezende, and D. Rosenbaum. From data to functa: Your data point is a function and you can treat it like one. In *International Conference on Machine Learning (ICML)*, 2022.
- [6] W. Cho, K. Lee, D. Rim, and N. Park. Hypernetwork-based meta-learning for low-rank physics-informed neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.